MICROBLOG ON A HOME SERVER

An end-to-end system design and operations handbook

A detailed, step-by-step explanation of Git, Python, Docker, MySQL, startup reliability, Nginx, UFW, Tailscale, HTTPS, Flask security, diagnostics and end-to-end HTTP requests.

Scope: begins at application deployment; omits installation and partitioning of Ubuntu. Based on the setup discussed in this project, with unverified configuration clearly marked.

Prepared: 17 October 2026 | Audience: beginner-to-intermediate backend and infrastructure developer

Table of contents

Table of contents 2

1. Scope, current state and the big picture 6

1.1 Components and their jobs . . . 6

1.2 Overall request paths . . . 6

2. Application source control and development workspace 7

2.1 Cloning the Git repository . . . 7

2.2 Source tree and separation of concerns . . . 7

2.3 Working directly on the server . . . 7

3. Python environments versus container environments 7

3.1 The host virtual environment . . . 7

3.2 Docker has a separate Python runtime . . . 8

3.3 What does and does not get isolated? . . . 8

4. Docker architecture: images, containers, Compose 8

4.1 Docker daemon and CLI . . . 8

4.2 Dockerfile to image to container . . . 8

4.3 Compose service definitions . . . 9

4.4 Commands and semantics . . . 9

5. Secrets and configuration at deployment time 9

5.1 Environment file transfer . . . 9

5.2 How Compose substitutes variables . . . 9

5.3 Credential roles . . . 10

6. MySQL, Docker storage and schema migrations 10

6.1 External named volume . . . 10

6.2 Database lifecycle . . . 10
6.3 Docker network and service names . . . 10
6.4 Flask models to disk . . . 11
6.5 Flask-Migrate and boot.sh . . . 11

7. Health checks, dependencies and restart behaviour 11

7.1 MySQL health check . . . 11
7.2 Compose depends_on . . . 11
7.3 Restart policies . . . 12
7.4 Failure scenarios . . . 12

8. Ports, loopback, and Docker port publication 12

8.1 Initial mapping . . . 12
8.2 Restricted mapping . . . 12
8.3 Why MySQL is not published . . . 13

9. Nginx reverse proxy and virtual hosting 13

9.1 Why Nginx, if Gunicorn serves HTTP? . . . 13
9.2 Local-LAN server block . . . 13
9.3 sites-available, sites-enabled and reload . . . 13
9.4 Local Nginx vs backend access . . . 13

10. Firewalling and access boundaries 14

10.1 UFW intended policy . . . 14
10.2 Docker and firewall caveat . . . 14
10.3 What ss showed . . . 14

11. Tailscale private networking 14

11.1 Tailscale VPN versus OpenSSH . . . 14
11.2 Local network and tailnet coexist . . . 14

12. Tailscale Funnel and public HTTPS **15**

12.1 Funnel is different from the private VPN . . . 15

12.2 Initial tunnel and background persistence . . . 15

12.3 Dedicated HTTPS-origin loopback listener . . . 15

12.4 Funnel switch . . . 15

12.5 Single domain, multiple services . . . 16

13. HTTP scheme propagation, ProxyFix and cookies **16**

13.1 Why Flask may see HTTP . . . 16

13.2 ProxyFix . . . 16

13.3 Cookie attributes . . . 16

13.4 SECRET_KEY and debug mode . . . 16

14. Hostname filtering and Flask version compatibility **17**

14.1 Why TRUSTED_HOSTS could not be used . . . 17

14.2 Two Nginx server blocks . . . 17

14.3 Verify before trusting the change . . . 17

15. Complete end-to-end browser request walkthrough **17**

15.1 GET /login from the public Internet . . . 17

15.2 POST /login and database access . . . 18

15.3 Creating a post . . . 18

15.4 Failure responses . . . 18

16. Logs, resource checks and operational observability **18**

16.1 stdout, stderr and Docker logging driver . . . 18

16.2 Operational commands . . . 19

16.3 What we skipped . . . 19

17. Database backups and recovery: what happened **19**

17.1 Backup attempt 19

17.2 Restore-test outcome 19

17.3 Sensible future plan 20

18. Deferred services and future architecture **20**

18.1 Elasticsearch 20

18.2 LibreTranslate 20

18.3 Redis and RQ 20

18.4 What not to overbuild for a portfolio showcase 20

19. Troubleshooting playbook and common misconceptions **21**

20. Practical deployment and change-management workflow **21**

20.1 Editing Python code 21

20.2 Editing Nginx 21

20.3 Editing .env and Compose 22

20.4 Modifying database schema 22

21. Architecture audit checklist **22**

22. Glossary and quick mental model **23**

1. Scope, current state and the big picture

This document reconstructs the application hosting architecture, beginning with cloning the Microblog repository onto Ubuntu and continuing through public HTTPS access. It separates each responsibility: source control, runtime, networking, persistence, HTTP proxying, security, and operations. It does not claim the deployment is a production-grade or highly available service. It is a publicly demonstrable learning project.

1.1 Components and their jobs

Component	Role
Git/GitHub	Track and distribute application source code.
Python venv	Optional direct-development dependency isolation on host.
Docker Engine	Build and run containers; manage networks and volumes.
Docker Compose	Declare Microblog, MySQL and optional supporting services.
boot.sh + Gunicorn	Run migrations with retry and serve the Flask WSGI application.
Flask + SQLAlchemy + PyMySQL	HTTP/application logic, ORM and MySQL connectivity.
MySQL + mysql-data	Relational state and persistent database files.
Nginx	Forward local or Funnel-origin HTTP requests to Gunicorn.
UFW	Host firewall filtering; not a substitute for Docker port isolation.
Tailscale private network	Authorised remote SSH/administration.
Tailscale Funnel	Public HTTPS address for chosen HTTP service.

1.2 Overall request paths

```
PUBLIC: Browser --HTTPS:443--> Tailscale Funnel
        --> localhost:8081 Nginx --> localhost:8000 Docker mapping
        --> Gunicorn --> Flask --> Docker network --> MySQL --> mysql-data

LAN:    Browser --> 192.168.1.75:80 Nginx --> localhost:8000
        --> Gunicorn --> Flask --> MySQL

ADMIN:  Windows VS Code/SSH --> Tailscale VPN or LAN --> OpenSSH on Ubuntu
```

Status note: Flask/Gunicorn/MySQL, Docker Compose, Nginx local routing, private SSH over Tailscale, and the intended Funnel path were implemented in conversation. Final runtime validation of some later security additions (ProxyFix, cookie attributes in browser, hostname rejection) was not explicitly reported. Redis/RQ rate limiting, monitoring dashboard and successful backup recovery were postponed.

2. Application source control and development workspace

2.1 Cloning the Git repository

```
cd ~/projects
git clone git@github.com:Rikesh-Mandal/microblog-flask.git
cd microblog-flask
git remote -v
```

What happens: Git transfers tracked source files, commit objects and branch references to a working copy under /home/rikesh/projects/microblog-flask. GitHub is not the runtime host. Cloning does not provision services, create environment secrets, run database migrations, or start the application. Those are deployment tasks.

Why use SSH: Ubuntu has its own GitHub SSH key, independent of the Windows-to-Ubuntu SSH identity. The server authenticates to GitHub as a Git client; the developer authenticates to Ubuntu with OpenSSH. The repository remote is the SSH URL, not an HTTPS URL using a credential manager.

2.2 Source tree and separation of concerns

```
~/projects/microblog-flask/
  app/                 Flask package: routes, models, templates, static assets
  migrations/          Alembic / Flask-Migrate history
  microblog.py         Application entry module
  config.py           Flask configuration
  requirements.txt    Python package specifications
  Dockerfile          Image-build instructions
  boot.sh             Container startup script
  compose.yaml        Multi-service declarations
  .env                Private deployment configuration (not committed)
  .venv/              Optional host-side Python environment (not committed)
```

The repository should contain reusable definitions and implementation, while machine-specific credentials and generated runtime state remain outside version control. The .env file and .venv directory should be ignored by Git. Ignoring a file does not undo a prior accidental commit; compromised credentials require rotation.

2.3 Working directly on the server

VS Code Remote SSH lets Windows provide the editor interface while source files, terminals, Git operations and development tools run on Ubuntu. This differs from editing Windows files mounted remotely. Edits under ~/projects are actual changes to the server filesystem. A Git commit records changes locally; a Git push transmits commits to GitHub.

3. Python environments versus container environments

3.1 The host virtual environment

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
which python
```

A virtual environment isolates Python packages and executable entry points for direct execution and testing on Ubuntu. Activation primarily adjusts PATH; it does not create a container, isolate the kernel, or

run MySQL. The system Python remains managed by Ubuntu, a reason to avoid installing project dependencies globally with sudo pip.

3.2 Docker has a separate Python runtime

The Ubuntu host was reported as Python 3.12.3 while the Microblog container reported Python 3.11.17, Flask 2.2.5 and Werkzeug 2.3.8. This is expected because an image supplies its own filesystem and Python interpreter. The Docker build installs its own packages. The host .venv is not automatically consumed by the image and should not be copied into it.

3.3 What does and does not get isolated?

Mechanism	Isolates	Does not inherently provide
Python venv	Python package installation	Separate kernel, network or database
Docker container	Process/filesystem/network namespaces and runtime configuration	A second Linux kernel or strong VM-style security boundary
MySQL named volume	Database file lifetime from container lifetime	An off-machine backup

4. Docker architecture: images, containers, Compose

4.1 Docker daemon and CLI

The docker command-line client communicates with the Docker Engine daemon. The daemon creates containers, sets up network namespaces and port publication, manages image layers and volumes, and coordinates with containerd. Docker Compose is a higher-level client that reads a desired multi-service configuration and asks Docker Engine to implement it.

4.2 Dockerfile to image to container

```
Dockerfile --docker build--> Immutable-ish image layers
Image --docker compose up--> Running container instance
Container --logs/state--> Docker daemon and storage drivers
```

A Dockerfile may start from a slim Python base, set /app as the work directory, install dependencies and copy application code. The image is a packaged template. A running container is a process-based instance of that image, with a writable layer and explicit mounts. Rebuilding an image does not mutate an already-running container automatically: Compose may need to recreate that container.

4.3 Compose service definitions

```
services:
  microblog:
    build: .
    # depends_on, environment, ports, restart, logging ...
  mysql:
    image: mysql:latest
    # environment, volumes, healthcheck ...
  elasticsearch:
    # optional search service
  libretranslate:
    # optional translation service
```

This is an illustrative skeleton, not a replacement for the actual Compose file. Each service defines a desired runtime configuration. The Core stack started with MySQL and Microblog; Elasticsearch and LibreTranslate remained defined in Compose but were not part of the validated core deployment. Compose often creates a default bridge network whose DNS resolves service names.

4.4 Commands and semantics

Command	Meaning
docker compose config --services	List services without starting them.
docker compose up -d mysql	Create/start MySQL and dependencies in background.
docker compose up -d --build --no-deps microblog	Build Microblog and apply without starting related services.
docker compose ps	Inspect current service container states.
docker compose exec microblog flask routes	Execute a Flask CLI command in the running container.
docker compose logs -f microblog	Follow logged output from service containers.

5. Secrets and configuration at deployment time

5.1 Environment file transfer

```
scp "E:\projects\microblog-flask\.env" \
    rikesh@192.168.1.75:/home/rikesh/projects/microblog-flask/.env
chmod 600 ~/projects/microblog-flask/.env
```

SCP transfers the file over encrypted SSH. The source is on Windows and the target is on Ubuntu. The Ubuntu file permission mode 600 permits only its owner to read/write, apart from privileged users. Credentials are not made secure simply because they are in .env; they must not be leaked in logs, shell output, images, Git history or screenshots.

5.2 How Compose substitutes variables

```
environment:
  MYSQL_DATABASE: ${MYSQL_DATABASE}
  MYSQL_USER: ${MYSQL_USER}
  MYSQL_PASSWORD: ${MYSQL_PASSWORD}
```

Compose substitutes ${NAME} using the available interpolation environment. The project .env is an interpolation input; to make variables available to a container they must be passed through environment, env_file or another explicit mechanism. A missing optional mail variable produced warnings and an empty substitution; this was deliberately left for later. Do not share fully resolved docker compose config output because it may reveal secrets.

5.3 Credential roles

Flask SECRET_KEY signs session data; it is not a MySQL password or SSH key. MySQL user credentials authenticate the application to the database. GitHub SSH keys authenticate Git operations. Windows SSH keys authenticate administrative login. Each credential should be generated, stored and rotated separately. Replacing SECRET_KEY can invalidate signed sessions.

6. MySQL, Docker storage and schema migrations

6.1 External named volume

The Compose project referenced an external volume named mysql-data. On a fresh Docker installation, docker compose up failed with “external volume mysql-data not found”. The external flag tells Compose not to create the volume; it must exist beforehand. The fix was an explicit volume creation.

```
docker volume create mysql-data
# Mapping inside the mysql service:
volumes:
  - mysql-data:/var/lib/mysql
```

The container sees /var/lib/mysql as its data directory; Docker persists those files under a named volume on the Ubuntu host. Recreating an application container leaves the database files intact. Recreating MySQL with the same compatible volume normally preserves data. Volume deletion, disk failure and logical data corruption can still destroy or damage data.

6.2 Database lifecycle

On first launch with an empty data directory, the official MySQL image initialises system tables and may create the configured database/user from environment variables. These initialisation values are not generally reapplied to an existing populated data directory. Changing MYSQL_PASSWORD in Compose does not automatically rewrite credentials stored in an existing database.

6.3 Docker network and service names

```
Microblog container --TCP 3306--> mysql (Compose DNS)

DATABASE_URL example:
mysql+pymysql://USER:PASSWORD@mysql:3306/microblog
```

In a container, localhost refers to that container itself. Because MySQL is in a separate container, the application uses mysql as the hostname. Docker DNS resolves that service name on the network. The app connects over a container network rather than requiring MySQL port 3306 to be published on Ubuntu.

6.4 Flask models to disk

A Flask route delegates data access to SQLAlchemy ORM. The ORM maps Python model operations to database statements; SQLAlchemy uses an engine and DBAPI driver, such as PyMySQL, to send SQL to MySQL. On commit, MySQL updates its transactional storage, writing into the mounted mysql-data directory. Example: creating a Post model instance, adding it to db.session and committing can insert a post row.

6.5 Flask-Migrate and boot.sh

```
#!/bin/bash
while true; do
    flask db upgrade
    if [[ "$?" == "0" ]]; then break; fi
    echo "Database upgrade failed; retrying..."
    sleep 5
done
exec gunicorn -b 0.0.0.0:8000 microblog:app
```

flask db upgrade applies versioned schema migrations (Alembic through Flask-Migrate). The loop retries until the command exits with zero status, tolerating MySQL startup delay. The shell special variable $? is the exit status of the immediately prior command; break exits the loop; sleep reduces retry pressure. exec replaces the shell with Gunicorn so the server becomes the container main process and receives lifecycle signals more directly.

Caveat: retrying every migration failure forever is not ideal: a malformed migration or wrong password is a permanent error. Production-grade code should discriminate transient failures and impose useful failure messages/retry budgets. Also, applying migrations on every application start is not always ideal in deployments with multiple replicas.

7. Health checks, dependencies and restart behaviour

7.1 MySQL health check

```
healthcheck:
  test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 --silent"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 30s
```

Docker periodically runs the test inside MySQL. A passing test marks the container healthy; sufficient counted failures make it unhealthy. start_period gives startup grace; interval sets check cadence; timeout bounds an individual check; retries counts consecutive failures. mysqladmin ping is primarily liveness, not proof that the Microblog application account can perform all required SQL operations.

7.2 Compose depends_on

```
depends_on:
  mysql:
    condition: service_healthy
```

When Compose starts the dependent service, it waits until MySQL passes its configured health check before starting Microblog. Contrast simple depends_on short syntax, which orders container startup but

does not require database readiness. It is not a continuously enforced dependency manager and cannot replace application retry logic.

7.3 Restart policies

```
restart: unless-stopped
```

The Docker daemon uses this policy to restart a container after unexpected termination and on appropriate daemon/host restarts, except when deliberately stopped. Compose does not have to remain running. A container marked unhealthy but still running is not normally restarted merely due to the health status. Docker restart policies do not enforce depends_on startup ordering during an ordinary host reboot.

7.4 Failure scenarios

Scenario	Likely outcome / mitigation
MySQL slower to initialise	Compose health check delays start when Compose orchestrates; boot.sh retries migrations.
Microblog process exits	Docker restart policy starts it again unless intentionally stopped.
MySQL becomes unhealthy but process runs	Status shows unhealthy; restart not automatic solely for this reason.
Host power cycle	systemd restarts Docker/Nginx/Tailscale; eligible containers restart; retry logic is helpful.
Broken database migration	boot.sh may loop forever; needs explicit error diagnosis.

8. Ports, loopback, and Docker port publication

8.1 Initial mapping

```
ports:
  - "8000:8000"
```

With no host bind IP specified, Docker normally publishes host port 8000 across host interfaces. Windows could open http://192.168.1.75:8000. Inside the container, Gunicorn listens on port 8000. The two 8000 values refer to different network namespaces, connected by Docker port mapping.

8.2 Restricted mapping

```
ports:
  - "127.0.0.1:8000:8000"
```

This is HOST_IP:HOST_PORT:CONTAINER_PORT. The host binds only its IPv4 loopback address. Nginx on Ubuntu can reach the backend at 127.0.0.1:8000, while a Windows PC connecting to 192.168.1.75:8000 should not reach that listener. Windows 127.0.0.1 refers to Windows, unless VS Code or another tunnel forwards the remote port.

8.3 Why MySQL is not published

Application containers connect to mysql:3306 over Docker's internal network. No host ports mapping is needed for this. Keeping port 3306 unpublished reduces the number of host-facing services. It does not obviate database authentication or correct user privileges.

9. Nginx reverse proxy and virtual hosting

9.1 Why Nginx, if Gunicorn serves HTTP?

Gunicorn already serves Flask through WSGI. Nginx introduces an HTTP reverse proxy at the host edge: it receives requests, selects a server block, forwards to a backend, and returns responses. Nginx can later manage multiple apps by hostname, serve static assets, implement traffic policies and terminate HTTPS if TLS is configured there. In this design HTTPS is instead terminated by Tailscale Funnel.

9.2 Local-LAN server block

```
server {
    listen 80;
    server_name 192.168.1.75;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

listen 80 opens the host HTTP entry point. server_name participates in virtual-server selection, not application startup. location / matches normal paths. proxy_pass sends the request to the backend on the Ubuntu host. Host preserves the requested hostname. X-Real-IP and X-Forwarded-For provide address metadata that must not automatically be trusted. X-Forwarded-Proto $scheme reflects the scheme on the incoming Nginx hop, which is HTTP for local access.

9.3 sites-available, sites-enabled and reload

```
sudo ln -s /etc/nginx/sites-available/microblog \
  /etc/nginx/sites-enabled/microblog
sudo unlink /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

The enabled directory is conventionally included by nginx.conf on Ubuntu. A symbolic link activates a site definition without duplicating the file. Unlinking the default enabled site stops serving the welcome page while retaining its source file. nginx -t validates syntax and configuration loading; reload applies changes gracefully. Avoid applying a broken configuration.

9.4 Local Nginx vs backend access

After installation, http://192.168.1.75 routes to Nginx on port 80, which proxies to port 8000. The application still runs independently inside Docker. The configuration filename "microblog" is only an administrative label: Nginx recognises the host, listen port, URL location and upstream address; it does not discover the Flask process from the filename.

10. Firewalling and access boundaries

10.1 UFW intended policy

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 80/tcp
# Initially also: sudo ufw allow OpenSSH
```

UFW is a host firewall frontend. Default-deny incoming reduces unsolicited inbound access; allow outgoing permits package downloads and other initiated connections. SSH must be allowed before enabling UFW remotely or a fresh connection can be blocked. We later discussed narrowing SSH to 192.168.1.0/24 and to the tailscale0 interface; the exact resulting rule set should always be confirmed with sudo ufw status numbered.

10.2 Docker and firewall caveat

Docker's own forwarding/NAT rules can make Docker-published ports behave differently from typical UFW INPUT filtering. Therefore the localhost-only port mapping is a distinct and important control. IPv6 requires separate consideration: a service listening on [::]:22 may accept IPv6 unless firewall and upstream network controls prevent it.

10.3 What ss showed

```
0.0.0.0:22       OpenSSH listens on host IPv4 interfaces
[::]:22          OpenSSH listens on IPv6
0.0.0.0:80       Nginx listens on host IPv4 interfaces
127.0.0.1:8000   docker-proxy bound to host loopback
127.0.0.53:53     systemd-resolved local DNS stub
```

The ss output confirms bind addresses and listening processes; it does not by itself prove Internet reachability. The router, ISP, IPv6 policy, tailnet access controls and firewall rules determine actual access paths. VS Code Remote SSH also produced loopback-only listeners which were not conventional public service ports.

11. Tailscale private networking

11.1 Tailscale VPN versus OpenSSH

Tailscale connects authorised devices into an encrypted tailnet. OpenSSH still authenticates the user and establishes the remote shell. These are separate layers: Tailscale establishes reachability and a private path; SSH provides secure administrative login. The Windows client connects to Ubuntu via its Tailscale 100.x.y.z address, without exposing port 22 through router forwarding.

11.2 Local network and tailnet coexist

The home-server continues to use the private LAN IP 192.168.1.75; Tailscale assigns an additional overlay address. Both addresses refer to the same machine, accounts and projects. VS Code Remote SSH can maintain two host aliases, one for LAN and one for Tailscale. A successful SSH connection through the tailnet does not make a website publicly accessible to people outside it.

12. Tailscale Funnel and public HTTPS

12.1 Funnel is different from the private VPN

Tailscale Funnel intentionally publishes a chosen local HTTP service to users on the public Internet. Visitors do not need to belong to the tailnet. It provides a public *.ts.net HTTPS hostname without purchasing a separate domain or setting up router port forwarding. Once enabled, even an obscure demo URL should be treated as public.

12.2 Initial tunnel and background persistence

```
sudo tailscale funnel 80
# After permissions are approved:
sudo tailscale funnel --bg 80
sudo tailscale funnel status
```

Funnel forwards public traffic to a local HTTP destination, initially Nginx on port 80. The first invocation required enabling Funnel on the tailnet through the Tailscale approval URL. Background mode stores the publication configuration rather than tying it to the terminal. The server, tailscaled, Nginx and Microblog must all remain operational for the site to work.

12.3 Dedicated HTTPS-origin loopback listener

```
server {
    listen 127.0.0.1:8081;
    server_name YOUR-TAILSCALE-HOST.ts.net;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

The dedicated loopback listener differentiates Funnel-origin traffic from local HTTP traffic at port 80. Funnel terminates public TLS, then sends HTTP to localhost:8081. The listener sets X-Forwarded-Proto to https because the external browser used HTTPS even though Nginx receives HTTP. This listener must remain loopback-only and dedicated to trusted Funnel traffic; simply sending an HTTPS header is not itself proof of TLS.

12.4 Funnel switch

```
sudo tailscale funnel status
sudo tailscale funnel --https=443 off
sudo tailscale funnel --bg http://127.0.0.1:8081
sudo tailscale funnel status
```

The new upstream target sends public requests through the specialised Nginx listener. Verify the actual CLI behaviour and status against the installed Tailscale version. Turning Funnel off removes public publication but does not stop the private VPN, Nginx, MySQL or Flask.

12.5 Single domain, multiple services

If a custom domain is purchased later, multiple apps can share it via subdomains such as blog.example.dev, crm.example.dev and api.example.dev. DNS and ingress route hostnames; Nginx can select backend services using distinct server_name blocks. Buying one domain does not mean a separate domain purchase for each subdomain.

13. HTTP scheme propagation, ProxyFix and cookies

13.1 Why Flask may see HTTP

Public TLS ends at Funnel, not Gunicorn. Funnel and Nginx make local HTTP requests to downstream services. Without trusted metadata, Flask may derive request.scheme as http and generate incorrect absolute URLs or redirects. Nginx can supply X-Forwarded-Proto, but Flask needs middleware that trusts precisely the headers supplied by known proxies.

13.2 ProxyFix

```
from werkzeug.middleware.proxy_fix import ProxyFix

app.wsgi_app = ProxyFix(
    app.wsgi_app,
    x_for=0, x_proto=1, x_host=0, x_port=0, x_prefix=0
)
```

ProxyFix wraps the Flask WSGI application and interprets selected X-Forwarded headers. x_proto=1 trusts one proxy-supplied scheme value; the zeros decline trust in forwarded client IP, host, port and path prefix. The trusted proxy boundary is the immediate Nginx hop which sets the protocol header. Do not blindly trust forwarded IP information or arbitrary client-provided X-Forwarded headers.

13.3 Cookie attributes

```
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Lax"
```

Secure instructs browsers to send the session cookie only over HTTPS, so cookie-based login over the LAN HTTP URL may fail. HttpOnly prevents ordinary JavaScript cookie reads. SameSite Lax constrains many cross-site cookie scenarios but is not full CSRF protection. Python None means omit the SameSite setting; string "None" means the explicit SameSite=None cookie attribute. Browser developer tools are useful for verifying actual Set-Cookie behaviour.

13.4 SECRET_KEY and debug mode

Flask SECRET_KEY signs session information and should be random, secret and kept out of Git. Signed cookies are not automatically encrypted. DEBUG and TESTING should be false in the deployed application. Gunicorn must be used rather than Flask’s debugger-enabled development server. An installed slim image may lack ps; use docker compose top microblog from the host to inspect processes.

14. Hostname filtering and Flask version compatibility

14.1 Why TRUSTED_HOSTS could not be used

The container was reported as Flask 2.2.5 and Werkzeug 2.3.8. Flask's TRUSTED_HOSTS configuration arrived in Flask 3.1, so merely setting it in this older version does not enforce hostname checking. We therefore discussed host filtering in the dedicated Nginx listener rather than claiming Flask validated hostnames.

14.2 Two Nginx server blocks

```
server {
    listen 127.0.0.1:8081 default_server;
    server_name _;
    return 444;
}
server {
    listen 127.0.0.1:8081;
    server_name YOUR-TAILSCALE-HOST.ts.net;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

The default block rejects unexpected hostnames with Nginx's special 444 connection-close response; the named block serves the intended Funnel hostname. server_name _ is a conventional placeholder, not an actual wildcard mechanism. This filtering applies only at that Nginx listener. An alternate direct access path or another server block may need its own restrictions.

14.3 Verify before trusting the change

```
sudo nginx -t
sudo systemctl reload nginx
curl -I -H 'Host: YOUR-TAILSCALE-HOST.ts.net' http://127.0.0.1:8081/
curl -v -H 'Host: malicious.example' http://127.0.0.1:8081/
```

The valid host should produce an expected Flask response such as 200 or a legitimate redirect; the invalid host should be rejected by the default block. These runtime checks were proposed in the conversation, but their final observed results were not supplied. Avoid claiming controls are verified without testing them.

15. Complete end-to-end browser request walkthrough

15.1 GET /login from the public Internet

1. The browser resolves the public *.ts.net hostname and establishes HTTPS with the Funnel endpoint. 2. Funnel accepts and forwards the request to Ubuntu's configured localhost listener. 3. Nginx selects its matching port-8081 server block and forwards to localhost:8000. 4. Docker maps the host port to the Microblog container port. 5. Gunicorn receives the HTTP request and invokes the Flask WSGI application. 6. ProxyFix, if present and correctly configured, sets the request scheme using Nginx's trusted header. 7. Flask matches the /login endpoint, renders a form or processes the submitted credentials, and returns a response. 8. The response traverses Gunicorn, Nginx and Funnel back to the browser.

15.2 POST /login and database access

A submitted login form is handled by its Flask route. The application validates input and queries the User model through SQLAlchemy. The ORM constructs SQL and passes it through a MySQL DBAPI driver, connecting to mysql:3306 on Docker networking. MySQL looks up the user and returns the result. The application verifies credentials and, on success, signs session information using SECRET_KEY. The browser receives a session cookie with the configured cookie attributes. The precise form handler and CSRF implementation must be reviewed in source code rather than inferred from infrastructure alone.

15.3 Creating a post

An authorised user sends a POST to the relevant application endpoint. Flask validates and processes the input, creates or modifies a Post model, and commits the transaction through SQLAlchemy. The MySQL container stores changed rows in data files backed by mysql-data. A successful HTTP response is returned. The Docker volume provides durability against container recreation, not against all forms of data loss.

15.4 Failure responses

If Nginx cannot connect to Gunicorn, a 502 Bad Gateway may appear. If the application raises an exception, Flask/Gunicorn can produce a 500 error, and messages may appear in Docker logs. If the database is unavailable during startup, boot.sh retries migrations; if MySQL disappears after Gunicorn is running, normal request handling must handle connection errors. Invalid hostnames at the Funnel listener may be closed with a 444 response.

16. Logs, resource checks and operational observability

16.1 stdout, stderr and Docker logging driver

Gunicorn and Flask commonly write to stdout/stderr. Docker captures those streams using a configured logging driver. We configured Docker’s local driver with bounded retention, rather than leaving logs to grow indefinitely. This is useful for inspecting startup errors and HTTP or application exceptions.

```
logging:
  driver: "local"
  options:
    max-size: "10m"
    max-file: "3"
```

The driver rotates files around 10 MB and retains a configured number of log files per container. The local driver compresses rotated logs by default. Changing a logging configuration commonly requires the container to be recreated; verify the applied container settings with docker inspect, not just by reading Compose YAML.

16.2 Operational commands

Command	Diagnostic purpose
docker compose logs --tail=30 microblog	Recent application logs.
docker compose logs -f mysql	Follow database logs.
docker compose ps -a	Find running/stopped containers.
docker stats --no-stream	CPU, memory and I/O snapshot.
sudo systemctl status nginx	Inspect reverse-proxy service.
sudo nginx -t	Validate reverse-proxy config.
sudo tailscale funnel status	Confirm Funnel target and publication.
sudo ufw status numbered	Inspect host firewall policy.
sudo ss -tulpen	See bound network ports and processes.

16.3 What we skipped

We discussed deeper monitoring with Prometheus, cAdvisor, Node Exporter and Grafana but elected not to install it. Docker stats and logs provide manual visibility, not persistent time-series monitoring or automated alerting. We also skipped tuning unattended upgrades and did not finish scheduled database backups.

17. Database backups and recovery: what happened

17.1 Backup attempt

We prepared a private host directory ~/backups/microblog and used mysqldump to create a logical SQL export. A logical dump contains SQL statements to recreate selected database structures/data rather than raw InnoDB database files. Credentials and backups are sensitive, so directory permissions were restricted.

17.2 Restore-test outcome

We attempted an isolated temporary MySQL container restoration. The expected microblog database did not appear when listing databases in the test instance. The root cause was not established, and the activity was postponed at the user’s request. Therefore the backup must be considered unverified, not a demonstrated recovery capability. Persistence and backup should never be conflated.

17.3 Sensible future plan

A reliable recovery plan should generate timestamped dumps with checked exit codes, protect incomplete/failed dumps from being mistaken for good ones, verify restore into a separate compatible MySQL instance, use retention rules and copy encrypted backups off the server. These are recommendations for later rather than claims of completed work.

18. Deferred services and future architecture

18.1 Elasticsearch

The Compose file defines Elasticsearch for full-text search, and the Microblog project has previously integrated search functionality. It was intentionally not included in the minimal verified home-server launch. An Elasticsearch container adds memory and disk requirements; its service configuration, credentials and named volume need to be validated separately before enabling public features.

18.2 LibreTranslate

LibreTranslate is present in the Compose definition along with the libretranslate-models volume, which is intended to persist downloaded models. It was not essential to validating the core Flask/MySQL deployment and was therefore postponed. Availability of its container is different from whether application translation routes are operational.

18.3 Redis and RQ

Redis was postponed for a future phase. It can provide a central store for counters used by rate limiting, and act as a broker/data store for Redis Queue (RQ) background jobs. A queue worker is a separate running process or container which reads jobs, executes tasks and records outcomes. A Redis-backed limiter is preferable to a simple in-process counter when multiple Gunicorn workers need shared state.

18.4 What not to overbuild for a portfolio showcase

Kubernetes, full monitoring stacks, elaborate load balancing and multi-region high availability are unnecessary for the stated goal. Correct API authentication/authorisation, reasonably current dependencies, clear documentation, modest resource limits and avoiding real sensitive user data are higher priorities for an Internet-accessible demo.

19. Troubleshooting playbook and common misconceptions

Symptom / misconception	Explanation and first check
Nginx is installed, so Flask is started by Nginx	False: Docker/Gunicorn run Flask independently. Nginx only forwards HTTP requests.
Host localhost:8000 still responds	Expected when Docker binds 127.0.0.1:8000. Test LAN-IP:8000 from another device.
Docker Compose file declares a volume but start fails	external: true requires creating the volume separately. docker volume ls.
MySQL is running but Flask starts too early	Running differs from healthy; depends_on for Compose start and boot.sh for retry.
Flask CLI says Python 3.11 but Ubuntu says 3.12	Container and host have independent runtimes.
No ps inside app container	Minimal images may omit it. Use docker compose top microblog from host.
TLS works in browser but Flask generates HTTP links	Check Funnel termination, dedicated Nginx entry, forwarded protocol and ProxyFix.
Invalid hostname still served	Confirm which Nginx listener accepted request and test exact host matching.
Backup file exists but restore has no microblog DB	Do not assume success; inspect dump contents, import stderr, exit code and schema.

20. Practical deployment and change-management workflow

20.1 Editing Python code

```
cd ~/projects/microblog-flask
# Edit code, run tests, inspect git diff
docker compose up -d --build --no-deps microblog
docker compose ps
docker compose logs --tail=30 microblog
```

When application files are baked into the image, use --build to get new code inside the container. Applying the new container may cause a short interruption. Preserve volumes. Validate functionality through the public HTTPS hostname and local Nginx path.

20.2 Editing Nginx

```
sudo nginx -t
sudo systemctl reload nginx
curl -I http://127.0.0.1
```

Editing a host-side Nginx configuration generally requires no Flask image build. Test syntax before reload, then verify expected host routing. A valid syntax check does not ensure the configured upstream application is alive.

20.3 Editing .env and Compose

New values that must enter a container environment require container recreation. Compose substitution and application image build-time configuration are separate. If a setting is copied into the image from source, rebuild; if it is injected via environment, recreate with the new environment. Avoid exposing docker compose config or docker inspect output that contains credentials.

20.4 Modifying database schema

Model changes and database migrations must be kept consistent. Generate, review and test migration scripts before deployment. The boot.sh retry loop applies pending migrations on startup but is not a substitute for validating migrations and preserving a recoverable backup, especially for destructive schema changes.

21. Architecture audit checklist

Area	What is confirmed / what needs attention
Code	Repository cloned over GitHub SSH; develop in VS Code Remote SSH.
Runtime	Dockerised Flask/Gunicorn and MySQL running; host .venv separate.
Network	Nginx port 80; Docker application published on loopback:8000.
State	mysql-data volume persists database files.
Startup	Restart policies, health check and boot.sh retry implemented.
Public HTTPS	Tailscale Funnel discussed/configured; verify current status/target after latest changes.
Flask proxy security	Verify ProxyFix at runtime and actual HTTPS redirects; host restrictions after later edits.
Session security	Verify Secure/HttpOnly/SameSite in browser dev tools.
Dependencies	Container Flask 2.2.5 / Werkzeug 2.3.8 reported; controlled upgrade pending.
Data recovery	Restore not verified; no reliable backup claim.
API security	Full write-route authorisation assessment not performed.
Redis / RQ / limiter	Deferred intentionally.

22. Glossary and quick mental model

Term	Definition
Reverse proxy	HTTP intermediary forwarding requests to upstream services.
WSGI	Python interface between an application server such as Gunicorn and Flask.
Image	Packaged runtime filesystem/template used to create containers.
Container	Running isolated process environment instantiated from an image.
Named volume	Docker-managed persistent storage outside container writable layer.
Bridge network	Docker network connecting containers with service-name DNS.
Loopback	Network interface representing the machine itself, e.g. 127.0.0.1.
Health check	Test recording whether an application responds as expected.
TLS termination	Point where an encrypted HTTPS connection is decrypted.
Forwarded header	Metadata added by proxies; only trusted proxy data should affect security-sensitive request state.
Tailnet	Private Tailscale network of authorised devices.
Funnel	Tailscale capability publishing selected services over public HTTPS.
Migration	Versioned database schema change applied through a migration tool.

Remember: Git stores your code. Compose defines services. Docker runs them. Gunicorn serves Flask. Flask implements your app. SQLAlchemy speaks to MySQL. MySQL writes persistent data to a named volume. Nginx directs HTTP requests to Gunicorn. Tailscale VPN provides private administration, while Funnel publishes selected HTTPS traffic. UFW and application controls create additional security boundaries.

Document accuracy: Command examples and configuration fragments are teaching aids and should be compared with your current repository before copying. The presence of a documented step does not mean every end-state verification was observed. No credentials are included.